

# AtendeMax — Documentação Técnica

## Sistema de Fila de Atendimento com Containerização Docker

---

### 1. Descrição Geral da Aplicação

O **AtendeMax** é um sistema web para gerenciamento de filas de atendimento presencial. A aplicação permite cadastrar clientes, organizá-los em uma fila com prioridade para atendimento preferencial, chamar o próximo da fila, cancelar atendimentos e registrar o histórico de atendimentos concluídos ou cancelados.

#### Funcionalidades principais

- **Cadastro de clientes** com nome e tipo (normal ou preferencial).
- **Fila de atendimento** ordenada automaticamente: clientes preferenciais têm prioridade sobre os normais; dentro de cada grupo, a ordem é por horário de chegada.
- **Chamada do próximo** cliente da fila, com controle de atendimento em andamento.
- **Cancelamento** de clientes que ainda aguardam na fila.
- **Conclusão de atendimento** para clientes em atendimento.
- **Histórico** com filtros por tipo e status.

#### Arquitetura

A solução é composta por três camadas containerizadas:

| Camada   | Tecnologia            | Responsabilidade                     |
|----------|-----------------------|--------------------------------------|
| Frontend | HTML, CSS, JavaScript | Interface do usuário no navegador    |
| Backend  | Python / FastAPI      | API REST e regras de negócio da fila |
| Banco    | PostgreSQL 16         | Persistência dos dados de clientes   |

O repositório está organizado em monorepo, com os diretórios `frontend/` e `backend/` na raiz do projeto, orquestrados por um único arquivo `docker-compose.yml`.

---

## 2. Mapeamento dos Containers

A aplicação é executada em **três containers** Docker, cada um com função específica:

### Container `atendemax-db` (serviço `db`)

| Item          | Valor                                                    |
|---------------|----------------------------------------------------------|
| Imagem base   | <code>postgres:16-alpine</code>                          |
| Porta exposta | <code>5432</code> (host) → <code>5432</code> (container) |
| Função        | Banco de dados relacional PostgreSQL                     |
| Dados         | Tabela <code>clientes</code> (fila e histórico)          |
| Persistência  | Volume Docker <code>postgres_data</code>                 |

### Container `atendemax-backend` (serviço `backend`)

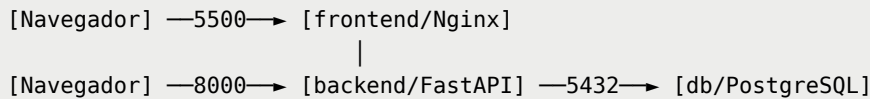
| Item             | Valor                                                                                      |
|------------------|--------------------------------------------------------------------------------------------|
| Imagem base      | <code>python:3.12-slim</code> (build local)                                                |
| Porta exposta    | <code>8000</code> (host) → <code>8000</code> (container)                                   |
| Função           | API REST FastAPI (uvicorn)                                                                 |
| Conexão ao banco | <code>postgresql://atendemax:atendemax123@db:5432/atendemax</code>                         |
| Endpoints        | <code>/fila</code> , <code>/clientes</code> , <code>/historico</code> , <code>/docs</code> |

### Container `atendemax-frontend` (serviço `frontend`)

| Item            | Valor                                                            |
|-----------------|------------------------------------------------------------------|
| Imagem base     | <code>nginx:alpine</code> (build local)                          |
| Porta exposta   | <code>5500</code> (host) → <code>80</code> (container)           |
| Função          | Servidor web de arquivos estáticos (Nginx)                       |
| Comunicação API | Requisições do navegador para <code>http://localhost:8000</code> |

## Rede e comunicação

Todos os containers compartilham a rede bridge `atendemax-network`:



O frontend é acessado pelo usuário em `http://localhost:5500`. As chamadas à API partem do navegador para `http://localhost:8000`. O backend conecta ao banco pelo hostname interno `db`, resolvido pela rede Docker.

### 3. Comandos Principais

Todos os comandos abaixo devem ser executados na **raiz do repositório** (`atendemax-docker/`).

#### Validar configuração do Compose

```
docker compose config
```

#### Construir e iniciar o ambiente completo

```
docker compose up --build
```

Para executar em segundo plano:

```
docker compose up --build -d
```

#### Verificar containers em execução

```
docker ps
```

#### Parar os containers

```
docker compose down
```

#### Parar e remover volumes (apaga dados do banco)

```
docker compose down -v
```

## Visualizar logs

```
docker compose logs -f
docker compose logs -f backend
```

## Acessos após subir o ambiente

| Recurso             | URL                        |
|---------------------|----------------------------|
| Interface web       | http://localhost:5500      |
| Documentação da API | http://localhost:8000/docs |
| Health check da API | http://localhost:8000/     |

# 4. Configuração dos Containers e docker-compose.yml

## Estrutura do repositório

```
atendemax-docker/
├── docker-compose.yml      # Orquestração dos 3 serviços
├── backend/
│   ├── Dockerfile
│   ├── .dockerignore
│   ├── main.py
│   ├── requirements.txt
│   ├── database/
│   └── services/
├── frontend/
│   ├── Dockerfile
│   ├── .dockerignore
│   ├── nginx.conf
│   ├── index.html
│   ├── css/
│   └── js/
└── docs/
    └── DOCUMENTACAO.pdf
```

## docker-compose.yml — visão geral

O arquivo define três serviços ( `db` , `backend` , `frontend` ), um volume nomeado ( `postgres_data` ) e uma rede compartilhada ( `atendemax-network` ).

**Serviço `db`** : utiliza a imagem oficial PostgreSQL 16 Alpine. As credenciais são definidas por variáveis de ambiente ( `POSTGRES_DB` , `POSTGRES_USER` , `POSTGRES_PASSWORD` ). Um healthcheck com `pg_isready` garante que o banco esteja pronto antes do backend iniciar.

**Serviço `backend`** : realiza build a partir de `./backend/Dockerfile` . Recebe a variável `DATABASE_URL` apontando para o host `db` na rede interna. Depende do serviço `db` com condição `service_healthy` .

**Serviço `frontend`** : realiza build a partir de `./frontend/Dockerfile` . Expõe a porta 5500 no host mapeada para a porta 80 do Nginx. Depende do serviço `backend` para garantir ordem de inicialização.

## Dockerfile do Backend

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

O `.dockerignore` exclui `venv/` , `__pycache__/` e `.pytest_cache/` para reduzir o contexto de build.

## Dockerfile do Frontend

```
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/
COPY css/ /usr/share/nginx/html/css/
COPY js/ /usr/share/nginx/html/js/
COPY assets/ /usr/share/nginx/html/assets/
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

O Nginx escuta na porta 80, serve `index.html` como página principal e entrega arquivos estáticos (CSS, JS, assets).

## CORS

O backend aceita requisições originadas de `http://localhost:5500` e `http://127.0.0.1:5500`, permitindo que o frontend containerizado se comunique com a API sem bloqueio de política CORS.

## Persistência de dados

O volume `postgres_data` é montado em `/var/lib/postgresql/data` no container do banco. Dados de clientes cadastrados permanecem após `docker compose down` e são restaurados em um novo `docker compose up`. Para limpar os dados, use `docker compose down -v`.

---

## 5. Fluxo de Validação (Teste Integrado)

1. Executar `docker compose up --build` na raiz do projeto.
2. Acessar `http://localhost:5500` e cadastrar um cliente.
3. Verificar se o cliente aparece na fila com posição correta.
4. Acessar `http://localhost:8000/docs` e testar os endpoints.
5. Executar `docker compose down` e depois `docker compose up` — confirmar que os dados persistem.
6. No DevTools do navegador, verificar ausência de erros de CORS.

---

*Documento gerado para entrega do projeto AtendeMax — Containerização com Docker Compose.*